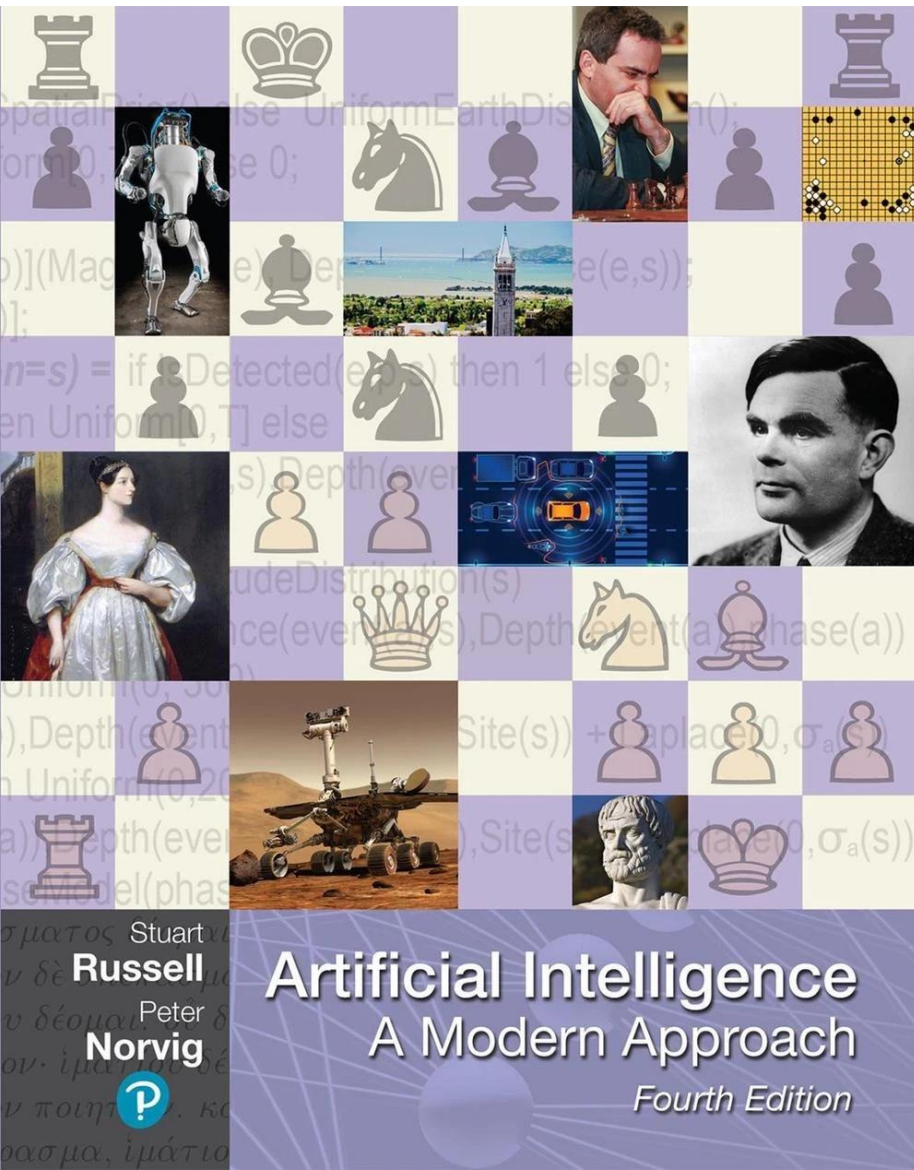
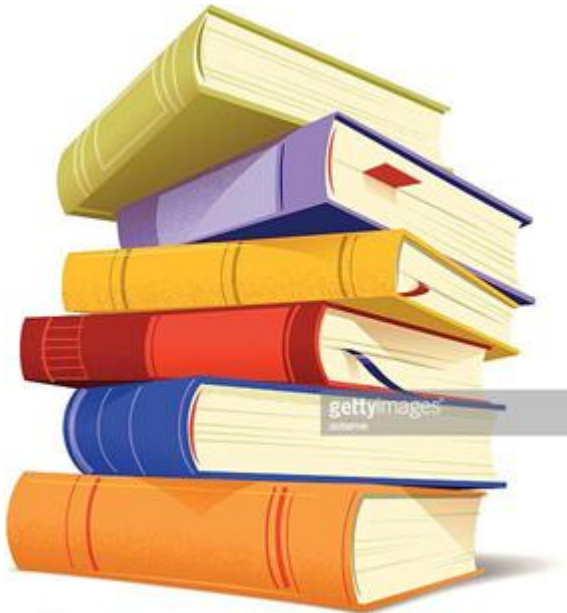


Chapter 8

Reinforcement Learning



Reinforcement Learning – Part I



- Introduction to Reinforcement Learning
- Model-Based Learning
- Model-Free Learning
- Passive Model-Free Learning

Introduction to Reinforcement Learning



What is Reinforcement Learning?

- **Definition:** Reinforcement learning (RL) is a framework where an **agent learns to behave** in an environment by receiving periodic **rewards or punishments**.
- **Contrast with Supervised Learning:** In RL, there is no "teacher" providing correct labels or actions; the agent learns from its own experience through **trial and error**.
- **The Reward Signal:**
 - Provides feedback on success (e.g., +1 for winning a game, -1 for losing).
 - Can be **sparse** (only at the end) or **shaped** (intermediate rewards for progress).
- **Objective:** The agent's goal is to **maximize the expected sum of future rewards**.

Policy

- A policy defines how the agent behaves.
- It specifies which action to take in each state.
- Policies can be deterministic or stochastic.
- Learning often focuses on finding an optimal policy.

Value Functions

- A value function estimates how good a state is.
- It represents expected future reward.
- **The further a reward is in the future, the more it is discounted.**
- The value depends on the policy being followed.
- Value functions guide decision making.

Model-Based Learning



Introduction to Model-Based Learning

- **Definition:** A model-based agent attempts to learn or is equipped with a **transition model** $P(s'|s, a)$ and a **reward function** $R(s, a, s')$.
- **Core Logic:** Instead of just learning what to do in each state, the agent learns how the environment works to help interpret reward signals and make decisions.
- **Structure:**
 - **Learning:** Updates the transition model based on observed outcomes.
 - **Planning:** Uses the learned model to compute an optimal policy through internal simulations or dynamic programming.
- **Contrast:** Unlike model-free agents, model-based agents have a means of "looking into the future".

Learning the Transition and Reward Models

- **Model Acquisition:** Since the environment is fully observable, learning the model is a **supervised learning task**.
- **Estimating Transitions:** The agent maintains counts $N_{s'|sa}$ of how often state s' is reached when action a is executed in state s .
- **Probability Calculation:** The transition model is estimated directly from these counts using **Maximum Likelihood Estimation:**

$$P(s'|s, a) = \frac{N_{s'|sa}}{N_{sa}}$$

- **Learning Rewards:** The reward function $R(s, a, s')$ is simply recorded as the reward signal received for each observed transition.

Adaptive Dynamic Programming (ADP)

- **Mechanism:** The agent learns that $P(s'|s,a)$, and $R(s,a,s')$. The agent then uses these models to calculate the state value.
- **Bellman Equation for Fixed Policy (π):** The agent plugs the learned transition and reward models into the Bellman equations for a fixed policy:

$$U^\pi(s) = \sum_{s'} P(s'|s, \pi(s)) [R(s, \pi(s), s') + \gamma U^\pi(s')]$$

(The value of the current state = total expected current reward + future value)

- **Active ADP:** In an active learning setting, the agent solves the full Bellman equation with the **max operator**:

$$U(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma U(s')]$$

(The agent will choose the action 'a' that yields the greatest expected value)

- **Solving Methods:** These can be solved using **Value Iteration** or **Policy Iteration**.

Exploration in Model-Based Learning

- **The Greedy Trap:** A **greedy agent** follows the policy it currently believes is optimal but often fails to learn the true utilities of unexplored areas.
- **Exploration Function:** To avoid suboptimality, agents use an **exploration function** $f(u, n)$ to trade off **greed** (preference for high utility u) against **curiosity** (trying actions with low count n).
- **Optimistic Priors:** The agent behaves initially as if unexplored states have "wonderful rewards" (R^+), encouraging it to visit them.
- **GLIE Property:** Effective schemes should be **Greedy in the Limit of Infinite Exploration**, ensuring all state-action pairs are tried infinitely often to find the optimal policy.

Efficiency and Computational Trade-offs

- **Data Efficiency:** Model-based methods (ADP) are highly **data-efficient**: they learn the true optimal policy with fewer trials than model-free methods (TD).
- **Computational Cost:** The main drawback is that ADP requires solving large sets of equations, which can be **computationally intractable** for vast state spaces.
- **Comparison with TD (Temporal Difference learning):**
 - **TD:** Simple, low computation per observation, but ignores connections between states.
 - **ADP:** More complex, high computation, but makes optimal use of environment structure.
- **Hybrid Approach:** Using a learned model to generate **pseudoexperiences** (simulated transitions) can bridge the gap.

Prioritized Sweeping

- **Problem:** After an observation changes the model ($P(s'|s,a)$, $R(s,a,s')$) slightly, running **full value iteration** is wasteful.
- **Concept:** Focus computation on updating the utilities of states that are most affected by the change.
- **Heuristic: Prioritized Sweeping** prefers making adjustments to states whose likely successors have just undergone a large utility change.
- **Benefit:** Allows approximate ADP to handle much larger state spaces while still learning nearly as fast as full ADP.

Strengths and Limitations

- Strengths:
 - Model-based methods allow planning and prediction.
 - They can adapt quickly with fewer samples.
- Limitations:
 - Building accurate models is difficult.
 - Computational cost can be high.

Model-Free Learning



Introduction to Model-Free Learning

- **Definition:** A model-free agent is one that **neither knows nor learns a transition model** $P(s'|s, a)$ of the environment.
- **Core Concept:** Instead of figuring out "how the world works," the agent learns a more **direct representation** of how to behave based on experience.
- **Key Varieties:**
 - **Action-utility learning (e.g. Q-learning):** Learning the "quality" of a specific action in a state $Q(s, a) \rightarrow$ choosing $a = \arg \max_a Q(s, a)$.
 - **Policy search:** Learning a mapping directly from states to actions $\pi(s) \rightarrow a$ without calculating utilities.
- **Benefit:** Highly applicable in **complex domains** where building an accurate transition model is impossible or too expensive.

Model-Free vs. Model-Based Learning

■ Model-Based (ADP):

- Learns $P(s'|s, a)$ and $R(s, a, s')$.
- Uses dynamic programming to solve the Bellman equations.
- **Data-efficient** but computationally heavy.

■ Model-Free (TD/Q-Learning):

- Avoids learning $P(s'|s, a)$ entirely.
- Updates estimates based on the **temporal difference** between successive states.
- **Computationally simple** per step but requires more trials to converge.

Passive Model-Free Learning



Introduction to Passive Model-Free RL

- **Passive Reinforcement Learning:** The agent's policy π is **fixed**, and its task is to learn the utility function $U^\pi(s)$.
- EX: $\pi(s_1) = a_1, \quad \pi(s_2) = a_3$
- **Model-Free Approach:** The agent does **not know** the transition model $P(s'|s, a)$ or the reward function $R(s, a, s')$, nor does it attempt to learn them.
- **Goal:** Learn the expected total discounted reward for each state ($U^\pi(s)$) while simply following the prescribed policy (π).
$$r_1 + \gamma r_2 + \gamma^2 r_3 + \dots$$
- **Agent Perception:** The agent only observes: $s \rightarrow s', \quad \text{reward} = r$
- Based on such experiences, the agent gradually updates the state utility.

Direct Utility Estimation

- **Core Idea:** Treat the utility of a state as the **expected reward-to-go** from that state onward.
- **Process:**
 - Each trial provides a sample of the total reward for each state visited.
 - The agent maintains a running average of these samples for each state in a table.
- **Supervised Learning:** This reduces reinforcement learning to a standard supervised learning problem where examples are **(state, reward-to-go)** pairs.
- **Limitation:** It ignores the fact that utilities of neighboring states are connected via the **Bellman equations**.

What is a reward-to-go?

- Let's assume an episode is as follows:

$$s1 \rightarrow s2 \rightarrow s3 \rightarrow \textit{terminal}$$

- Rewards received:

$$r1 = 2, r2 = 3, r3 = 5$$

- Then the reward-to-go at:

$$s1 = 2 + 3 + 5 = 10, s2 = 3 + 5 = 8, s3 = 5$$

- The agent will store these values as a template to learn the utility.

Temporal-Difference (TD) Learning

- TD Learning updates the value of the current state by comparing it to the next state. Commonly used formula:

$$U(s) \leftarrow U(s) + \alpha[r + \gamma U(s') - U(s)]$$

- Where:
 - $(U(s))$: current utility of state (s)
 - (r) : reward just received
 - $(U(s'))$: utility of the next state
 - (γ) : discount factor, level of interest in the future
 - (α) : learning rate, learning speed

Temporal-Difference (TD) Learning

- **Mechanism:** Adjusts utility estimates so they agree with the local constraints imposed by the **Bellman equations**.
- **Key Insight:** Use the observed transitions to adjust the utilities of states to be consistent with their successors.
- **No Model Required:** Unlike Adaptive Dynamic Programming (ADP), TD does not need a transition model; the environment **supplies the connections** through observed transitions.
- **Efficiency:** TD is a crude but efficient first approximation to ADP, requiring much less computation per observation.

The TD Update Equation

- **The Rule:** Whenever a transition occurs from s to s' via action $\pi(s)$, the utility is updated: $U^\pi(s) \leftarrow U^\pi(s) + \alpha[R(s, \pi(s), s') + \gamma U^\pi(s') - U^\pi(s)]$
- **Parameters:**
 - α : The learning rate (step size).
 - γ : The discount factor.
- **Error Signal:** The term $[R + \gamma U(s') - U(s)]$ is the **TD error**, representing the difference between the current estimate and the one provided by the successor.
- **Convergence:** $U^\pi(s)$ converges to the correct value if α decreases over time.

TD vs. ADP

Features	ADP	TD
Model Usage	Learns the model $P(s' s, a)$ and solves the MDP.	Model-free; adjusts estimates based on single transitions.
Computation	Computationally heavy; involves solving simultaneous equations.	Very simple; one basic arithmetic update per step.
Learning Efficiency	More data-efficient (learns faster from fewer trials).	Slower to converge and shows higher variability.

